Tugas Analisis Multimedia: Audio, Gambar, Video

Mata Kuliah: Sistem & Teknologi Multimedia
Nama: Muhammad Fauzi Azizi
NIM: 122140106

Deskripsi Tugas

Tugas ini bertujuan untuk memberikan pengalaman praktis kepada mahasiswa dalam meng□implementasikan teknologi Remote Photoplethysmography (rPPG) yang telah dibahas di kelas. Mahasiswa diminta untuk membangun sebuah sistem perangkat lunak yang mampu mendeteksi detak jantung seseorang secara real-time menggunakan kamera (webcam) tanpa kontak fisik.

Lingkup Pekerjaan

Mahasiswa diharapkan untuk melakukan hal-hal berikut:

- 1.*Rekreasi Pipeline rPPG:* Mengimplementasikan kembali proses pemrosesan sinyal rP□PG seperti yang didemonstrasikan di kelas, yang mencakup tahapan:
- Deteksi Wajah: Menggunakan pustaka seperti OpenCV atau MediaPipe untuk mendeteksi dan melacak wajah (ROI).
- Ekstraksi Sinyal: Melakukan spatial averaging pada area kulit (khususnya kanal Hijau/Green) untuk mendapatkan sinyal mentah.
- Pemrosesan Sinyal: Menerapkan teknik detrending (misalnya sliding average) dan Bandpass Filter (rentang 0.67 Hz - 4.0 Hz) untuk membersihkan noise.
- Estimasi BPM: Menggunakan Transformasi Fourier (FFT) untuk mencari frekuensi dominan dan mengonversinya menjadi Beats Per Minute (BPM) atau dapat juga menggunakan fungsi findpeaks dari scipy.

In [ ]:
```
import cv2
import mediapipe as mp
import numpy as np
from collections import deque
from scipy.signal import find_peaks
from scipy.fft import rfft, rfftfreq
import time
from scipy.signal import butter, filtfilt, detrend # Tambahan import

# ------------------------
# DISPLAY / OVERLAY (edit these)
# ------------------------
```

```
# Colors are in BGR (OpenCV)
COLOR_BPM = (0, 0, 255)       # red for BPM
COLOR_GPM = (255, 0, 0)       # blue for GPM label/value
COLOR_CONST = (0, 255, 255)   # yellow for constants
COLOR_BG = (0, 0, 0)          # background rectangle color
FONT = cv2.FONT_HERSHEY_SIMPLEX

GPM_LABEL = "GPM"
CONST_LABEL = "CONST_X"
CONST_VALUE = 1.234

# -------------------------
# rPPG / algorithm params (edit if needed)
# -------------------------
WINDOW_SECONDS = 15.0         # sliding window (seconds)
FPS_FALLBACK = 30.0
BPF_LOW = 0.8                 # Hz (~40 BPM)
BPF_HIGH = 2.5                # Hz (~240 BPM)
FILTER_ORDER = 4
DETREND_MA_SEC = 2.0
MOTION_THRESHOLD = 0.03       # reject frame if face center moves > fraction of fac
MIN_SKIN_PIXELS = 30
CONFIDENCE_MIN = 1.2

# -------------------------
# Skin segmentation thresholds (tweak for lighting / skin tones)
# HSV (H:0-180)
HSV_LOWER = np.array([0, 30, 40], dtype=np.uint8)
HSV_UPPER = np.array([25, 255, 255], dtype=np.uint8)
# YCrCb
YCrCb_LOWER = np.array([0, 133, 77], dtype=np.uint8)
YCrCb_UPPER = np.array([255, 173, 127], dtype=np.uint8)

# -------------------------
# MediaPipe init
# -------------------------
mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh(static_image_mode=False,
                             max_num_faces=1,
                             refine_landmarks=False,
                             min_detection_confidence=0.5,
                             min_tracking_confidence=0.5)

# -------------------------
# ROI helper (left cheek)
# -------------------------
def bbox_from_landmarks(landmarks, img_w, img_h, padding=0.03):
    xs = [lm.x for lm in landmarks]
    ys = [lm.y for lm in landmarks]
    x_min = max(int(min(xs) * img_w), 0)
    x_max = min(int(max(xs) * img_w), img_w - 1)
    y_min = max(int(min(ys) * img_h), 0)
    y_max = min(int(max(ys) * img_h), img_h - 1)
    w = x_max - x_min
    h = y_max - y_min
    px = int(w * padding)
```

```
    py = int(h * padding)
    x1 = max(x_min - px, 0)
    y1 = max(y_min - py, 0)
    x2 = min(x_max + px, img_w - 1)
    y2 = min(y_max + py, img_h - 1)
    return x1, y1, x2, y2

# FUNGSI INI DIMODIFIKASI UNTUK MENGGESER ROI KE BAWAH
def left_cheek_roi_from_bbox(bbox, fraction_w=0.28, fraction_h=(0.35, 0.70)):
    # Nilai fraction_h diubah dari (0.20, 0.60) menjadi (0.35, 0.70)
    # 0.35 menggeser batas atas (top) ROI lebih jauh ke bawah, menjauhi mata.
    x1,y1,x2,y2 = bbox
    W = x2 - x1
    H = y2 - y1
    rw = int(W * fraction_w)
    top = int(y1 + H * fraction_h[0]) # Batas atas baru (sekitar 35% dari atas waja
    bot = int(y1 + H * fraction_h[1]) # Batas bawah baru (sekitar 70% dari atas waj
    left_x1 = x1 + int(W * 0.08)
    left_x2 = left_x1 + rw
    left = (max(left_x1,0), max(top,0), min(left_x2, x2), min(bot, y2))
    return left

# -------------------------
# Skin segmentation utilities
# -------------------------
def skin_mask_combined(roi_bgr):
    if roi_bgr.size == 0:
        return None
    hsv = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2HSV)
    mask_hsv = cv2.inRange(hsv, HSV_LOWER, HSV_UPPER)
    ycrcb = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2YCrCb)
    mask_ycrcb = cv2.inRange(ycrcb, YCrCb_LOWER, YCrCb_UPPER)
    mask = cv2.bitwise_and(mask_hsv, mask_ycrcb)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3,3))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=1)
    return mask

def mean_green_skin(roi_bgr, use_skin=True):
    if roi_bgr.size == 0:
        return None
    if not use_skin:
        return float(np.mean(roi_bgr[:,:,1]))
    mask = skin_mask_combined(roi_bgr)
    if mask is None:
        return float(np.mean(roi_bgr[:,:,1]))
    mask_bool = mask.astype(bool)
    count = np.count_nonzero(mask_bool)
    if count < MIN_SKIN_PIXELS:
        return float(np.mean(roi_bgr[:,:,1]))
    greens = roi_bgr[:,:,1][mask_bool]
    return float(np.mean(greens))

# -------------------------
# MAIN realtime loop
# -------------------------
```

```
def main():
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("ERROR: cannot open webcam")
        return

    fps_cam = cap.get(cv2.CAP_PROP_FPS)
    fs = fps_cam if fps_cam and fps_cam > 1 else FPS_FALLBACK
    buffer_size = int(WINDOW_SECONDS * fs)
    signal_buf = deque(maxlen=buffer_size)
    time_buf = deque(maxlen=buffer_size)
    prev_center = None
    prev_width = None

    # Hitung koefisien filter
    b_coeff, a_coeff = butter_bandpass(BPF_LOW, BPF_HIGH, fs, FILTER_ORDER)

    print("Starting rPPG (left cheek + skin seg). Press 'q' to quit.")

    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                print("No frame; exiting.")
                break

            img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            results = face_mesh.process(img_rgb)
            h, w, _ = frame.shape
            use_frame = True

            if results.multi_face_landmarks:
                lm = results.multi_face_landmarks[0].landmark
                bbox = bbox_from_landmarks(lm, w, h, padding=0.03)
                x1,y1,x2,y2 = bbox
                center = ((x1+x2)/2.0, (y1+y2)/2.0)
                width = (x2-x1)
                if prev_center is not None and prev_width is not None:
                    dx = abs(center[0]-prev_center[0]) / prev_width
                    dy = abs(center[1]-prev_center[1]) / prev_width
                    if dx > MOTION_THRESHOLD or dy > MOTION_THRESHOLD:
                        use_frame = False
                prev_center = center; prev_width = width

                # Panggilan fungsi ini menggunakan nilai default baru (0.35, 0.70)
                # yang sudah disesuaikan di definisinya di atas.
                # Jika ingin menimpanya lagi, gunakan:
                # lx1, ly1, lx2, ly2 = left_cheek_roi_from_bbox(bbox, fraction_h=(0

                # Biarkan seperti ini agar menggunakan default yang sudah dimodifik
                lx1, ly1, lx2, ly2 = left_cheek_roi_from_bbox(bbox)
                left_roi = frame[ly1:ly2, lx1:lx2]

                if use_frame:
                    g = mean_green_skin(left_roi, use_skin=True)
                    if g is not None:
```

```
                    signal_buf.append(g)
                    time_buf.append(time.time())

                # draw left cheek ROI
                cv2.rectangle(frame, (lx1,ly1), (lx2,ly2), (0,255,0), 2)
                if not use_frame:
                    cv2.putText(frame, "MOTION - frame rejected", (10,60), FONT, 0.
            else:
                cv2.putText(frame, "No face detected - show face fully", (10,30), F

            # Estimate BPM when buffer has enough data (>= 60% of window)
            bpm_text = "BPM: --"
            conf_text = ""
            method_text = ""
            if len(signal_buf) >= int(0.6 * buffer_size):
                sig = np.array(signal_buf)
                bpm_fft, conf_fft = estimate_bpm_fft(sig, fs, b_coeff, a_coeff)
                bpm_pk, conf_pk = estimate_bpm_peaks(sig, fs)
                chosen_bpm = None; chosen_conf = None; method = None
                if bpm_fft and conf_fft and conf_fft >= CONFIDENCE_MIN:
                    chosen_bpm = bpm_fft; chosen_conf = conf_fft; method = "FFT"
                elif bpm_pk and conf_pk:
                    chosen_bpm = bpm_pk; chosen_conf = conf_pk; method = "Peaks"
                elif bpm_fft and conf_fft:
                    chosen_bpm = bpm_fft; chosen_conf = conf_fft; method = "FFT-low
                if chosen_bpm is not None:
                    bpm_text = f"BPM: {chosen_bpm:.1f}"
                    conf_text = f"Conf: {chosen_conf:.2f}"
                    method_text = f"Method: {method}"

            # --- OVERLAY: left-side text (edit here) ---
            # optional small dark background for readability
            cv2.rectangle(frame, (5,5), (300,130), COLOR_BG, -1)
            # BPM (color editable above)
            cv2.putText(frame, bpm_text, (10,30), FONT, 0.8, COLOR_BPM, 2)
            # Show last raw green mean as GPM (or replace with any metric you want)
            if len(signal_buf) > 0:
                last_g = signal_buf[-1]
                cv2.putText(frame, f"{GPM_LABEL}: {last_g:.2f}", (10,60), FONT, 0.7
            else:
                cv2.putText(frame, f"{GPM_LABEL}: --", (10,60), FONT, 0.7, COLOR_GP
            # Const label/value

            # right-side small info
            if conf_text:
                cv2.putText(frame, conf_text, (320,30), FONT, 0.6, (255,255,255), 1
            if method_text:
                cv2.putText(frame, method_text, (320,55), FONT, 0.6, (200,200,200),

            cv2.imshow("rPPG - Left Cheek (skin seg)", frame)
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                break

    except KeyboardInterrupt:
        print("\nStopped by user (KeyboardInterrupt). Exiting...")
```

```
    finally:
        cap.release()
        cv2.destroyAllWindows()
        print("Resources released. Bye.")


# -------------------------
# FILTER & ESTIMATION FUNCTIONS
# -------------------------

def butter_bandpass(lowcut, highcut, fs, order=4):
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return b, a

def sliding_detrend(sig, fs, ma_seconds=DETREND_MA_SEC):
    n = len(sig)
    window = int(max(1, round(ma_seconds * fs)))
    if n < window + 1:
        return detrend(sig)
    kernel = np.ones(window) / window
    ma = np.convolve(sig, kernel, mode='same')
    return sig - ma

def estimate_bpm_fft(sig, fs, b_coeff, a_coeff, low=BPF_LOW, high=BPF_HIGH):
    n = len(sig)
    if n < 4:
        return None, None
    sig_d = sliding_detrend(sig, fs)
    sig_d = (sig_d - np.mean(sig_d)) / (np.std(sig_d) + 1e-8)
    try:
        filtered = filtfilt(b_coeff, a_coeff, sig_d)
    except Exception:
        return None, None
    yf = np.abs(rfft(filtered))
    xf = rfftfreq(n, 1.0/fs)
    idx = np.where((xf >= low) & (xf <= high))
    if len(idx[0]) == 0:
        return None, None
    xf_band = xf[idx]; yf_band = yf[idx]
    peak_idx = np.argmax(yf_band)
    freq_peak = xf_band[peak_idx]
    bpm = freq_peak * 60.0
    confidence = yf_band[peak_idx] / (np.median(yf_band) + 1e-6)
    return bpm, confidence

def estimate_bpm_peaks(sig, fs):
    n = len(sig)
    if n < 4:
        return None, None
    sig_d = sliding_detrend(sig, fs)
    smooth = np.convolve(sig_d, np.ones(3)/3, mode='same')
    min_dist = int(max(1, round(fs * 0.25)))
```

```
    peaks, _ = find_peaks(smooth, distance=min_dist)
    if len(peaks) < 2:
        return None, None
    intervals = np.diff(peaks) / fs
    avg_itv = intervals.mean()
    bpm = 60.0 / avg_itv if avg_itv > 0 else None
    expected = (n / fs) * 1.5
    confidence = len(peaks) / (expected + 1e-6)
    return bpm, confidence

# --------------------------
if __name__ == "__main__":
    main()
```

```
Starting rPPG (left cheek + skin seg). Press 'q' to quit.

Stopped by user (KeyboardInterrupt). Exiting...
Resources released. Bye.
```

Penelitian rPPG oleh bapak telah meletakkan dasar dengan menggunakan detektor Dlib pada data video offline. Namun, saya menawarkan peningkatan signifikan melalui dua inovasi utama. Pertama, saya beralih ke detektor MediaPipe yang dioptimalkan untuk performa, memungkinkan sistem beroperasi dalam mode realtime. Perubahan ini menjadikan aplikasi lebih relevan dan dapat diterapkan dalam skenario praktis. Kedua, dan yang paling krusial, saya mengadopsi pendekatan analisis ROI (Region of Interest) yang sangat spesifik, yaitu membatasi pemrosesan hanya pada area pipi kiri. Strategi ini tidak hanya meningkatkan efisiensi komputasi tetapi juga memungkinkan ekstraksi fitur yang lebih terfokus dan berpotensi lebih akurat untuk sinyal.

REFERENSI

saya dibantu dengan Ai GPT : https://chatgpt.com/share/692d1b1d-6154-8006-b0a7-2567548d96cc